DroneArjuna backend code guide

This guide explains the DroneArjuna application in beginner friendly terms, including both its React frontend and FastAPI backend. It gives a map of the main folders and follows the important paths between the user interface, API, databases, drone connections, and supporting services. It also explains several core backend modules: `backup.py`, `audit_middleware.py`, `hf_feed.py`, `search.py`, and `storage.py`.

1. Big picture

The backend is a Python application built with FastAPI. It exposes HTTP and WebSocket endpoints to the frontend, uses SQLAlchemy to read and write databases, and runs background services for telemetry and integrations.

```
Browser / frontend
       |
       | HTTP JSON / WebSocket
       v
FastAPI application (app/main.py)
  | middleware: security headers, CORS, mutation audit
  | routers: auth, drone control, flight, inventory, system, ...
  v
Services and core helpers
  |                        |                        |
  v                        v                        v
PostgreSQL              TimescaleDB                External services
users, missions,         latest telemetry +         Redis, RabbitMQ,
inventory, audit         telemetry history          Elasticsearch, MinIO

Drone / vessel devices ---- MAVLink or UDP/NMEA ----> backend
```

`main.py` is the composition root: it creates the FastAPI app, installs middleware, includes routers, and starts/stops long-lived services. A router handles a family of URLs. A service/helper contains logic that routes or background tasks call. The database models describe stored rows.

2. Request path through FastAPI

For a typical API call, the path is:

1. The frontend sends an HTTP request, for example `POST /api/flight/missions`.
2. FastAPI middleware wraps the request. CORS controls which browser origins may call the API; security middleware adds response headers; audit middleware records eligible state changes.
3. FastAPI finds the matching route in an included router, validates inputs (often with Pydantic), and resolves dependencies such as the current user or database session.
4. The route calls SQLAlchemy and/or a service module.
5. FastAPI serializes the returned Python value to JSON and sends the response.

Middleware is like a wrapper around every request. A route is the function that handles a particular URL and HTTP method. A dependency is a reusable function FastAPI runs to provide a value, such as a database session or authenticated user.

3. Python and FastAPI concepts used here

- **Module/import:** Each `.py` file is a module. `from app.core.search import ...` imports a name from another module. This lets the code group related responsibilities.
- **Class and instance:** `HFFeedListener` and `DumpTarget` are classes; `hf_feed` and `MAIN_TARGET` are configured objects made from those classes.
- **Type hints:** `query: str`, `-> list[dict]`, and `int | None` describe expected values. They help readers and tools; FastAPI also uses endpoint type hints for request validation and API documentation.
- **Async/await:** `async def` defines a coroutine. `await` pauses it while network or database I/O is pending, allowing the server to handle other work. `asyncio.create_task(...)` starts background work without waiting for it to finish first.

- **Context manager:** `async with ...` reliably opens and closes resources such as database sessions, even if an error occurs.
- **Decorators:** `@app.get(...)`, `@app.middleware(...)`, and `@event.listens_for(...)` register a function with FastAPI or SQLAlchemy. The decorated function is still ordinary Python code, but the framework calls it at the right time.
- **Dependency injection:** `Depends(...)` in a route asks FastAPI to provide a shared value. This avoids manually constructing a session or checking access in every endpoint.
- **Pydantic model:** A `BaseModel` describes and validates structured request data. For example `RestoreRequest` limits `database` to the allowed literal strings and rejects extra fields.
- **Exception handling:** `try/except` handles failures locally. These modules generally log failures so optional integrations (like search indexing) do not take down the main API.
- **Global client cache:** Variables such as `_client` hold a lazily created client so each helper call can reuse it. `close_client()` releases it during shutdown.
- **Callback / protocol class:** `asyncio.DatagramProtocol` methods are called by asyncio when UDP data arrives. `_NMEAProtocol.datagram_received` is a callback, not a function called directly by a route.
- **Environment configuration:** `get_settings()` reads application settings, usually sourced from environment variables. This keeps URLs, credentials, and feature switches out of hard-coded logic.

4. The five core modules

`app/core/backup.py` — *database backup and recovery*

Purpose: Save database table contents as timestamped JSON dumps and restore data when configured checks detect empty tables. Dumps are kept locally and uploaded to MinIO. Main PostgreSQL and telemetry TimescaleDB have separate `DumpTarget` configurations.

Main concepts and flow:

1. `DumpTarget` groups the database metadata, session factory, local folder, and MinIO bucket for one database.
2. `create_dump()` reads rows through SQLAlchemy, converts values such as datetimes to JSON-compatible strings, writes a dump file, and uploads it to MinIO.
3. `_pack_dump()` signs the canonical JSON with HMAC-SHA256 and can encrypt it with Fernet when an encryption key is configured. `_unpack_dump()` decrypts if needed and checks the signature before restore.
4. `restore_latest_dump()` loads recent dumps, skips invalid ones, and restores into the target database. `restore_all()` runs restoration for both targets at startup.
5. `run_backup_scheduler()` periodically creates full backups. `run_integrity_monitor()` periodically checks whether expected tables became empty. SQLAlchemy commit hooks also schedule a debounced incremental dump after writes.

The main database backup includes registered model tables. Telemetry backups include only the latest-state tables (`telemetry` and `telemetry_gauges`), not the rapidly growing history tables.

In simple terms: this module takes snapshots, stores a durable copy, verifies the copy before trusting it, and can refill tables after data loss.

`app/core/audit_middleware.py` — *audit state-changing API requests*

Purpose: Record who made a state-changing API request, the route and category, related drone/mission IDs when detectable, client IP, and HTTP result.

Request flow: `audit_mutations(request, call_next)` first calls the next layer to let the endpoint run. Once it gets the response, it filters for `POST`, `PUT`, `PATCH`, or `DELETE` under `/api/`, skipping selected auth paths. It verifies the user's JWT to extract the user ID, categorizes the path, and calls `record_mutation()` to write an audit row with a separate database session.

It logs the route template and path/status, not the request body or query string. That avoids recording sensitive payloads and single-use query tokens. Audit errors are caught and logged so an audit database problem does not replace the endpoint response.

`app/core/hf_feed.py` — *vessel position over UDP/NMEA*

Purpose: Receive NMEA position sentences over UDP and publish the latest vessel position to Redis.

Data flow:

```
Vessel GPS/NMEA -> UDP datagram -> source/rate/size checks -> NMEA parse
    -> coordinates/speed validation -> Redis key `vessel:position` (10 second TTL)
```

`HFFeedListener.start()` creates the Redis client and UDP endpoint. For each datagram, `_NMEAProtocol` checks the source IP against an allowlist and limits message size/rate, then schedules async ingestion. `ingest_nmea()` parses the sentence, validates values (including optional maximum speed), and updates Redis. `stop()` closes the UDP transport and Redis client.

The short Redis TTL means consumers can treat an expired value as stale or unavailable. In `main.py`, the listener starts as part of secondary service initialization and is stopped during graceful shutdown.

`app/core/search.py` — *Elasticsearch indexing and search*

Purpose: Keep searchable copies of drone type, payload type, and threat-system records in Elasticsearch.

`get_client()` lazily creates a shared asynchronous Elasticsearch client. `ensure_indices()` creates indices and field mappings. The `index_*` functions write one document; `delete_document()` removes one. `bulk_index_all(db)` reads existing active catalog records from PostgreSQL on startup and indexes them. `search_inventory(query)` runs a fuzzy multi-field search and returns result dictionaries tagged as drone, payload, or threat.

Elasticsearch is a search index, not the authoritative database. The code catches and logs indexing/search errors and returns an empty result list on failed search, so an Elasticsearch outage should not prevent normal database-backed routes from working.

`app/core/storage.py` — *MinIO object storage*

Purpose: Store analyst artifacts such as source images, generated videos, and PDF reports in an S3-compatible MinIO bucket.

`classify_content_type()` maps allowed MIME types to artifact kinds and rejects others. `ensure_bucket()` creates the bucket if needed and attempts to configure browser CORS. `build_object_key()` makes a path under a job ID with a random UUID to avoid filename collisions. `upload_bytes()`, `get_presigned_url()`, and `delete_object()` perform the object operations.

The Python `boto3` client is synchronous, while these functions are async. `asyncio.to_thread(...)` runs the blocking boto3 call on a worker thread so it does not stall FastAPI's event loop. A presigned URL lets the browser access one object for a limited time without exposing MinIO credentials.

This storage helper is for application artifacts. `backup.py` also writes to MinIO, but uses dedicated database-backup buckets and its own backup format/signature workflow.

5. How startup and shutdown connect the pieces

`main.py` defines a FastAPI lifespan function. FastAPI runs the code before `yield` at startup and the code after `yield` at shutdown.

Startup, simplified:

1. Retry connections and create SQLAlchemy model tables for PostgreSQL and TimescaleDB.
2. Restore database contents from backups if needed.
3. Ensure a default admin exists.
4. Start optional integrations and workers: RabbitMQ consumers, MinIO bucket setup, telemetry recorder, UDP vessel feed, and Elasticsearch bulk indexing.
5. Start background tasks for auto-connect, periodic backups, and integrity checks.
6. Begin accepting requests.

Shutdown: cancel background tasks, stop the feed and telemetry recorder, close RabbitMQ and Elasticsearch, and dispose of both database engines.

The exact order matters: restore occurs before default admin seeding so a wiped database is not mistaken for a new installation.

6. Frontend architecture (React + TypeScript)

The frontend is a single page application. The browser loads `frontend/src/main.tsx`, which mounts the React app into the HTML page. `App.tsx` checks the login state and chooses the login, password setup, password reset, or signed-in application view. `AppShell.tsx` provides the shared sidebar, top bar, notifications, camera overlay, and the active workspace.

```
main.tsx
  ■■■ App.tsx                login / password setup / signed-in decision
       ■■■ AppShell.tsx      shared navigation and overlays
            ■■■ Fleet        list/connect/manage available drones
            ■■■ Plan         missions, map, waypoints, fleet assignment
            ■■■ Fly          live flight controls, map, instruments
            ■■■ Monitor      telemetry gauges/charts, logs, black box
            ■■■ Settings     users, drone/payload types, vessels, backup/security
```

Frontend folders

- `frontend/src/components/` contains reusable UI pieces. `layout/` holds login and app-shell elements; `common/` contains shared dialogs, badges, camera and error-boundary components; `instruments/` contains gauge displays.
- `frontend/src/workspaces/` contains the five main screens. A workspace composes smaller components for a complete user task.
- `frontend/src/api/` contains functions that call backend endpoints. `client.ts` is the shared Axios setup; files such as `droneControl.ts`, `droneFlight.ts`, and `auth.ts` group endpoint calls by feature.
- `frontend/src/store/` contains shared browser state using Zustand. Components subscribe to only the values they need, and store actions call the API or update local state.
- `frontend/src/utils/` contains reusable non-UI logic such as geofence calculations and route collision helpers.

Frontend concepts

- **React component:** A function that returns UI, usually written with JSX. Components can be nested and reused.
- **Props:** Inputs passed from a parent component to a child, such as the currently selected workspace.
- **State:** Values that can change while the UI is open. `useState` is component-local state; Zustand stores state shared across distant components.
- **Hooks:** Functions such as `useState` and `useEffect` that let React components keep state and react to lifecycle changes. `useEffect` is used, for example, to hydrate the login session on startup.
- **TypeScript:** JavaScript with type annotations. Interfaces such as the telemetry frame describe the shape of data expected from the API and WebSocket.
- **API client:** Axios provides HTTP calls and shared behavior such as cookies, timeouts, and retrying after a session refresh.
- **WebSocket:** A persistent two-way connection used for frequent updates. The telemetry store keeps a connection for live drone data; the system-events store listens for backend notices.

Frontend state stores

The stores are organized around application concepts rather than screen components:

Store	What it keeps
`authStore`	Current user/session state, login, MFA, and logout actions
`fleetStore`	Drone instances and fleet selection/status
`missionStore`	Mission data and planning state
`telemetryStore`	Live telemetry frames and per-drone WebSocket connections
`vesselStore`	Vessel data shown in fleet/mission workflows

Store	What it keeps
`notificationStore`	In-app notifications
`systemEventsStore`	Notices pushed by the backend over WebSocket
`eventLogStore`	User-facing event history
`connectionHealthStore`	Connection health and reconnect behavior
`timezoneStore`	Timezone preferences and formatting
`accessRequestStore`	Account access request state

Frontend-to-backend connection

`frontend/src/api/client.ts` configures Axios to use same-origin relative `/api` URLs, include cookies, and add the `X-Requested-With` header. The backend sets HttpOnly session cookies; browser JavaScript cannot read those cookie values. When a normal request gets a 401 response, the client attempts a session refresh and retries once. WebSockets cannot attach the usual HTTP headers, so the frontend first asks the backend for a short-lived, single-use ticket and then connects with that ticket.

During development, `vite.config.ts` proxies `/api` to the backend. It also proxies `/camera-api` to the native camera backend. That second path exists because a host webcam is not normally available inside the Dockerized backend; IP cameras can be accessed through either setup, but webcam device indices need the native process.

7. Backend architecture (FastAPI + SQLAlchemy)

The backend entry point is `backend/app/main.py`. It assembles the application from feature routers and shared core services. The code is organized as a modular monolith: one FastAPI app/process, with separate Python modules for feature areas and infrastructure concerns.

Backend folders

- `backend/app/main.py`: creates the FastAPI app, installs middleware, includes routers, and owns startup/shutdown.
- `backend/app/modules/`: feature areas. `drone_control/` handles MAVLink connections, commands, telemetry processing/recording, simulation and camera functions; `drone_flight/` handles missions, fleet assignment and geospatial/airspace checks; `drone_master/` manages drone and payload catalog data; `drone_inventory/` handles inventory/search functions. The Analyst module is optional in this checkout/runtime.
- `backend/app/core/`: cross-cutting services such as authentication, security/RBAC, audit, events, backups, storage, search, and system notices.
- `backend/app/models/`: SQLAlchemy database row definitions.
- `backend/app/schemas/`: Pydantic request/response shapes. Schemas validate data crossing the HTTP boundary; models describe persisted database rows. They often have similar fields but serve different jobs.
- `backend/app/utils/`: shared helpers for MAVLink, geography, execution and geofencing.
- `backend/alembic/`: versioned database schema changes (migrations).

Main backend feature areas

Feature area	Typical responsibility
Auth and security	Login/session, MFA, roles, access checks, password flows
Drone control	MAVLink connection lifecycle, commands, telemetry, camera stream
Drone flight	Missions, waypoints, geofences/airspace, fleet assignment
Drone master	Drone model/type and payload catalog configuration
Drone inventory	Inventory records and searchable catalog operations
System	Health, system event stream, backup list and operator restore

An endpoint normally lives in a `router.py`, uses a Pydantic schema for input/output, calls a service or helper for logic, then reads/writes an SQLAlchemy model or an external service. Not every route needs a separate service file; smaller route groups can keep simple logic near the router.

Backend persistence and supporting services

- **PostgreSQL + PostGIS:** Main relational data such as users, missions, drone configuration, inventory and audit records. PostGIS adds geographic database features.
- **TimescaleDB:** Time-series telemetry. Latest-state telemetry tables can be updated in place; high-frequency history is append-only and managed separately.
- **Redis:** Short-lived/current values and coordination, including the vessel position feed.
- **RabbitMQ:** Message broker used for asynchronous events and consumers.
- **Elasticsearch:** Search index for catalog-like drone, payload, and threat records; PostgreSQL remains the source of truth.
- **MinIO:** S3-compatible storage for analyst artifacts and database backup copies.
- **MAVLink:** Protocol used by the backend to communicate with supported flight controllers.

8. End-to-end examples

A. User opens the flight monitor

1. `App.tsx` confirms the session and renders `AppShell`.
2. The user selects Monitor in the sidebar; `AppShell` renders `MonitorWorkspace`.
3. The workspace reads the selected drone and telemetry state from Zustand.
4. The telemetry store requests a WebSocket ticket through the shared API client.
5. The browser opens the telemetry WebSocket; the backend validates its ticket and streams telemetry.
6. The store updates, React re-renders subscribed gauges/charts, and the backend recorder persists configured telemetry to TimescaleDB.

B. User creates or uploads a mission

1. A planning component collects mission fields and waypoints in React state/store.
2. A feature API function sends the request to the flight router.
3. FastAPI validates the body against a Pydantic schema and checks the user's role.
4. The route/service stores the mission through SQLAlchemy and returns a response schema.
5. When the user uploads the mission, drone-control logic sends the waypoint commands through the MAVLink manager to the flight controller.
6. Mutation audit middleware records the API action and outcome; explicit command audit events may add more detail.

C. Vessel location arrives

1. A vessel device sends an NMEA sentence by UDP.
2. `HFFeedListener` checks source/rate/size, parses and validates the position, then stores it briefly in Redis.
3. Backend services that need vessel position read the current Redis value and can display/use it in fleet or planning workflows.

9. Build and deployment picture

The README describes a Docker Compose stack with the frontend, FastAPI backend, PostgreSQL, TimescaleDB, Redis, RabbitMQ, MinIO and pgAdmin. The frontend is built with Vite and React; the backend runs with Uvicorn. Database migrations are managed by Alembic. The frontend and backend communicate through the Vite proxy during development and through the configured deployment routing in a deployed environment.

Useful commands documented by the project README:

```
cd DroneArjuna
docker compose up -d --build
docker compose exec backend alembic upgrade head
```

The frontend package scripts include `npm run dev` for local development and `npm run build` to type-check and create a production bundle.

10. Where to look next

If you want to understand...	Start here
Frontend startup and login routing	`frontend/src/main.tsx`, `App.tsx`
Main UI layout and screen switching	`frontend/src/components/layout/AppShell.tsx`
A particular screen	matching folder under `frontend/src/workspaces/`
HTTP calls and WebSocket tickets	`frontend/src/api/`
Shared UI state	matching store under `frontend/src/store/`
App setup and service lifecycle	`backend/app/main.py`
HTTP endpoints	`backend/app/modules/*/router.py` and `backend/app/core/*_router.py`
Database connections and sessions	`backend/app/database.py`
Database table definitions	`backend/app/models/`
Request/response validation	`backend/app/schemas/`
Login and JWT handling	`backend/app/core/auth.py` and `security.py`
Business logic	matching `service.py` or module service/helper files
Environment settings	`backend/app/config.py` and `.env.example`

For one user-visible feature, follow it in this order: **workspace/component** → **frontend store/API function** → **backend router** → **service/helper** → **schema/model** → **database or external client**. For startup services, follow `main.py` **lifespan** → **service** `start()` → **event/data processing** → `stop()`.